DOC-09 設計転換ガイド

対象: クラシックMVC経験者・先輩エンジニア / 読了目安: 7分

09. MVCからの設計転換ガイド

思考の転換・現場の割り切り・実践コード対比

従来の「Controller → Service → DAO」によるMVCモデルから、なぜ「ドメイン中心の自己防衛アーキテクチャ」へと舵を切ったのか。 理論の押し付けではなく、現場の課題解決と開発スピード（テスト容易性）の観点からその勘所を解説します。

Section 01

なぜ従来のMVCから設計を転換したのか？

初期の単純なCRUD機能では、従来の「Controller → Service → DAO」が直感的で素早い実装に適していました。 しかし、客先精算幅（140h〜180h）、36協定アラート、中抜け休憩、週報多層承認といった**複雑なSES業務ルール**が重なるにつれ、以下の壁に直面しました。

1. DB都合の侵入
ServiceがDAOのSQLやテーブル構造に引きずられ、テーブル仕様の都合が業務計算に漏れ出す。

2. 貧血JavaBeans
Beanがgetter/setterの箱になり、不正データ（実労働と内訳の不一致等）が防げずバグが多発。

3. テスト不能の悪循環
Serviceの検証にMySQLとTomcat起動が必須となり、単体テストが書かれずデグレの恐怖が増大。

✨ 転換の果実: 業務ルールを純粋Javaで孤立させたことで、**MySQL・Tomcat起動不要で20件のテストが0.8秒で完了**する強固な基盤が実現しました。

Section 02 直感で掴む設計思想：任天堂Switchとカセット

ヘキサゴナル／クリーンアーキテクチャの本質は、**「ゲーム機本体とカセット」**の関係です。

🎮 **Switch本体（コア）**
ゲームのルールそのもの（ドメイン/ユースケース）。TVが何であるか、操作機器が何かは知りません。

🕹️ **Joy-Con（入力アダプター）**
ユーザーの操作を受け取り本体に伝える（Web Action / REST API）。入れ替え可能。

📺 **TV/ドック（出力アダプター）**
本体が決めた映像端子の規格に合わせて表示・保存（MySQL / インメモリスタブ）。

💡 **依存性の逆転 (DIP):** 依存の矢印は必ず外側から中心へ向かいます。中核の業務ルールはMySQLに依存せず、外側のDAOが中核のインターフェースを実装します。

Section 03 「MVCのこれ、新アーキテクチャでどこ？」完全対応マップ

従来のMVC要素	新アーキテクチャでの名称	主な責務・役割	配置パッケージ
Action / Controller	Web Adapter (Controller)	HTTP受付、パラメータ変換、UseCase呼出、画面遷移/JSON	`controller/action/, api/`
Service (業務手順)	UseCase Service	ユースケースの進行役。ドメインを操作して保存	`usecase/attendance/ 等`
Service内の計算・判定	**Domain Model / Service**	純粋な計算・バリデーション・状態判定（外部依存ゼロ）	`domain/contract/, compliance/`
JavaBean (受動的データ箱)	**自己防衛 Entity**	自身の不変条件を自ら検証し、不正な状態を拒絶	`domain/attendance/ 等`
DAO (DBアクセス)	Repository (Adapter)	コアのInterfaceをJDBC/SQLで具象実装	`infrastructure/persistence/`

Section 04 コード対比で見る「思考の転換」（Before / After）

【Before】従来のMVC：ロジックがServiceやActionに散乱・テスト困難

```
public class KintaiService {
    private KintaiDao dao = new KintaiDao(); // DBに直結
    public void update(KintaiBean bean) throws Exception {
        int total = calc(bean.getStartTime(), bean.getEndTime());
        if (total != (bean.getDirect() + bean.getIndirect())) {
            throw new Exception("内訳不一致"); // 業務ルールがServiceに散乱
        }
        dao.update(bean); // DB起動必須
    }
}
```

【After】新アーキテクチャ：自己防衛エンティティ ＋ 依存性逆転

```
// 1. ドメイン層：自ら正しさを防衛（DailyAttendance.java）
public class DailyAttendance {
    public void fillAttendance(LocalTime start, LocalTime end, int breakMin, List<WorkTime
        if (this.isLocked) throw new IllegalStateException("月次確定済のため変更不可");
        this.startTime = start; this.endTime = end;
        validateSegmentsIntegrity(); // ★ 不正データはメモリ上に1ミリ秒たりとも存在できない！
    }
}

// 2. ユースケース層：手順の調停役（FillAttendanceUseCase.java）
public class FillAttendanceUseCase {
    private final AttendanceRepository repository; // インターフェースに依存
    public DailyAttendance execute(FillAttendanceInput input) {
        DailyAttendance att = repository.findByDate(input.getStaffId(), input.getDate());
        att.fillAttendance(input.getStart(), input.getEnd(), input.getBreakMin(), input.ge
        repository.save(att);
        return att; // そのまま返却
    }
}
```

Section 05 現場での現実的割り切り（教科書通りにやらない勇気）

割り切り1: Presenterは捨て、戻り値で返す

理論上のクリーンアーキテクチャにある「Output Boundary & Presenter」はWebでは複雑すぎるため、 **UseCase Interactor が結果を戻り値として直接返す**形に割り切っています。

割り切り2: スタブ先行開発で即時テスト

DB作成やSQL実装を待つ必要はありません。Repositoryのインターフェースに対して「Mapのスタブ」を用意すれば、 **DB接続前に業務ロジックのJUnitテストを先行完成**させられます。

Section 06 よくある疑問とFAQ（先輩エンジニアからの質問集）

Q1. クラス数やインターフェースが増えて冗長ではないですか？

初見ではファイル数が増えますが、1クラスの責務が極小化されるため、変更時の影響調査コストとバグ調査時間は激減します。

Q2. ドメインEntityとDAOのテーブルBeanで似たクラスができるのはなぜ？

業務ルールとDBテーブル構造の変更理由（正規化やインデックス都合）は異なるためです。分離によりDB変更が業務に波及しません。

Q3. 新機能・改修時はどこから書き始めるべきですか？

Domain（純粋Javaでルール作成＋単体テスト） → UseCase（手順組み立て） → Adapter（Web/JDBC） の順が鉄則です。